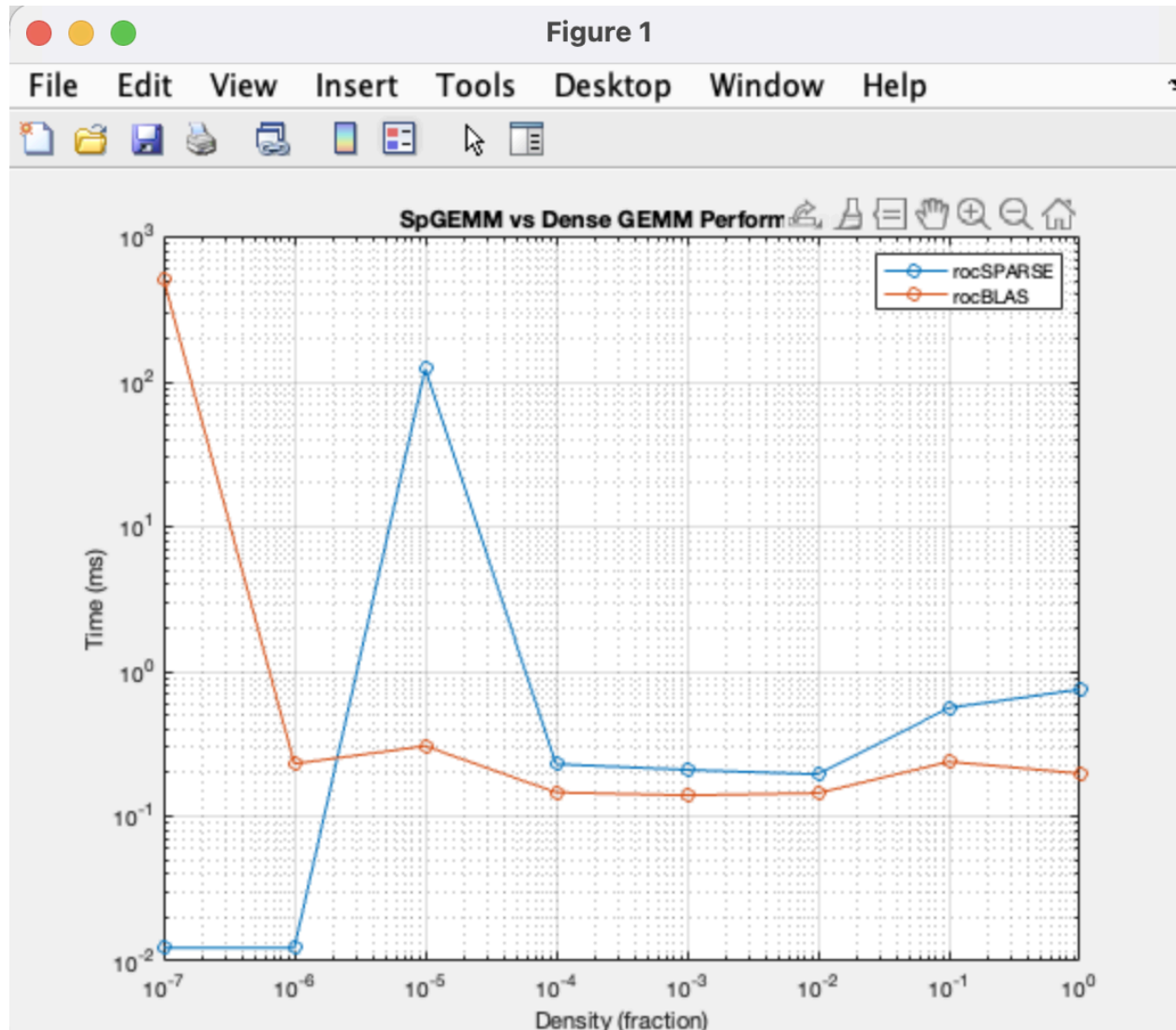


Brandon Nguyen, 827813045
COMPE596 - Spring 2026
4/11/2026

COMPE596 P09 - AMD HIP Sparse & Dense Multiplication With rocSPARSE and rocBLAS

1) Runtime Plot



2) Code Listing

// Main Program

```
#include <assert.h>
```

```
#include <hip/hip_runtime.h>
```

```

#include <math.h>
#include <roclblas/roclblas.h>
#include <rocsparse/rocsparse.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HIP_CHECK(call)
do {
    hipError_t err__ = (call);
    if (err__ != hipSuccess) {
        fprintf(stderr, "HIP error at %s:%d: %s\n", __FILE__, __LINE__,
            hipGetErrorString(err__));
        exit(EXIT_FAILURE);
    }
} while (0)

#define ROCBLAS_CHECK(call)
do {
    roclblas_status status__ = (call);
    if (status__ != roclblas_status_success) {
        fprintf(stderr, "rocBLAS error at %s:%d: %d\n", __FILE__, __LINE__,
            (int)status__);
        exit(EXIT_FAILURE);
    }
} while (0)

#define ROCSPARSE_CHECK(call)
do {
    rocsparse_status status__ = (call);
    if (status__ != rocsparse_status_success) {
        fprintf(stderr, "rocSPARSE error at %s:%d: %d\n", __FILE__,
            __LINE__, (int)status__);
        exit(EXIT_FAILURE);
    }
} while (0)

static float rand01(void) {
    return (float)rand() / (float)RAND_MAX;
}

```

```
static void zero_dense(float *a, int n) {
    memset(a, 0, (size_t)n * (size_t)n * sizeof(float));
}
```

```
static void fill_dense_with_density(float *a, int n, float density_fraction) {
    for (int i = 0; i < n * n; ++i) {
        float p = rand01();
        if (p < density_fraction) {
            a[i] = rand01();
        } else {
            a[i] = 0.0f;
        }
    }
}
```

```
static int dense_to_csr(const float *a, int n, int **row_ptr_out, int **col_ind_out,
                      float **val_out, int *nnz_out) {
    int *row_ptr = NULL;
    int *col_ind = NULL;
    float *val = NULL;
    int nnz = 0;

    row_ptr = (int *)malloc((size_t)(n + 1) * sizeof(int));
    if (row_ptr == NULL) {
        return 0;
    }

    row_ptr[0] = 0;
    for (int row = 0; row < n; ++row) {
        for (int col = 0; col < n; ++col) {
            float x = a[row * n + col];
            if (x != 0.0f) {
                ++nnz;
            }
        }
        row_ptr[row + 1] = nnz;
    }

    if (nnz > 0) {
```

```

    col_ind = (int *)malloc((size_t)nnz * sizeof(int));
    val = (float *)malloc((size_t)nnz * sizeof(float));
    if (col_ind == NULL || val == NULL) {
        free(row_ptr);
        free(col_ind);
        free(val);
        return 0;
    }
} else {
    col_ind = NULL;
    val = NULL;
}

int ptr = 0;
for (int row = 0; row < n; ++row) {
    for (int col = 0; col < n; ++col) {
        float x = a[row * n + col];
        if (x != 0.0f) {
            col_ind[ptr] = col;
            val[ptr] = x;
            ++ptr;
        }
    }
}

assert(ptr == nnz);
*row_ptr_out = row_ptr;
*col_ind_out = col_ind;
*val_out = val;
*nnz_out = nnz;
return 1;
}

static void csr_to_dense(int n, const int *row_ptr, const int *col_ind,
                        const float *val, float *dense_out) {
    zero_dense(dense_out, n);
    for (int row = 0; row < n; ++row) {
        for (int k = row_ptr[row]; k < row_ptr[row + 1]; ++k) {
            int col = col_ind[k];
            dense_out[row * n + col] = val[k];
        }
    }
}

```

```

    }
}
}

```

```

static void dense_gemm_host(const float *a, const float *b, float *c, int n) {
    zero_dense(c, n);
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < n; ++k) {
            float aik = a[i * n + k];
            if (aik == 0.0f) {
                continue;
            }
            for (int j = 0; j < n; ++j) {
                c[i * n + j] += aik * b[k * n + j];
            }
        }
    }
}

```

```

static float max_abs_diff(const float *x, const float *y, int count) {
    float max_diff = 0.0f;
    for (int i = 0; i < count; ++i) {
        float d = fabsf(x[i] - y[i]);
        if (d > max_diff) {
            max_diff = d;
        }
    }
    return max_diff;
}

```

```

static void free_host_csr(int **row_ptr, int **col_ind, float **val) {
    if (*row_ptr != NULL) {
        free(*row_ptr);
        *row_ptr = NULL;
    }
    if (*col_ind != NULL) {
        free(*col_ind);
        *col_ind = NULL;
    }
    if (*val != NULL) {

```

```

        free(*val);
        *val = NULL;
    }
}

int main(int argc, char **argv) {
    const int n = 213;
    const float tol = 1.0e-2f;
    const int start_exp = -5;
    const int end_exp = 2;

    int device_id = 0;
    if (argc >= 2) {
        device_id = atoi(argv[1]);
    }

    srand(12345);
    HIP_CHECK(hipSetDevice(device_id));

    rocblas_handle blas_handle = NULL;
    rocsparse_handle sparse_handle = NULL;
    ROCBLAS_CHECK(rocblas_create_handle(&blas_handle));
    ROCSPARSE_CHECK(rocsparse_create_handle(&sparse_handle));
    ROCSPARSE_CHECK(
        rocsparse_set_pointer_mode(sparse_handle, rocsparse_pointer_mode_host));

    float *hA = (float *)malloc((size_t)n * (size_t)n * sizeof(float));
    float *hB = (float *)malloc((size_t)n * (size_t)n * sizeof(float));
    float *hC_ref = (float *)malloc((size_t)n * (size_t)n * sizeof(float));
    float *hC_dense = (float *)malloc((size_t)n * (size_t)n * sizeof(float));
    float *hC_sparse_dense = (float *)malloc((size_t)n * (size_t)n * sizeof(float));

    if (hA == NULL || hB == NULL || hC_ref == NULL || hC_dense == NULL ||
        hC_sparse_dense == NULL) {
        fprintf(stderr, "Host allocation failed.\n");
        free(hA);
        free(hB);
        free(hC_ref);
        free(hC_dense);
        free(hC_sparse_dense);
    }
}

```

```

    ROCBLAS_CHECK(rocblas_destroy_handle(blas_handle));
    ROCSPARSE_CHECK(rocsparse_destroy_handle(sparse_handle));
    return EXIT_FAILURE;
}

```

```

printf("density_percent,density_fraction,nnz_a,nnz_b,rocsparse_ms,rocblas_ms,sparse
_max_abs_err,dense_max_abs_err,sparse_ok,dense_ok\n");

```

```

for (int exp10 = start_exp; exp10 <= end_exp; ++exp10) {
    float density_percent = powf(10.0f, (float)exp10);
    float density_fraction = density_percent / 100.0f;
    if (density_fraction > 1.0f) {
        density_fraction = 1.0f;
    }
}

```

```

fill_dense_with_density(hA, n, density_fraction);
fill_dense_with_density(hB, n, density_fraction);
dense_gemm_host(hA, hB, hC_ref, n);

```

```

int *h_row_ptr_A = NULL;
int *h_col_ind_A = NULL;
float *h_val_A = NULL;
int nnz_A = 0;

```

```

int *h_row_ptr_B = NULL;
int *h_col_ind_B = NULL;
float *h_val_B = NULL;
int nnz_B = 0;

```

```

if (!dense_to_csr(hA, n, &h_row_ptr_A, &h_col_ind_A, &h_val_A, &nnz_A) ||
    !dense_to_csr(hB, n, &h_row_ptr_B, &h_col_ind_B, &h_val_B, &nnz_B)) {
    fprintf(stderr, "CSR conversion allocation failed.\n");
    free_host_csr(&h_row_ptr_A, &h_col_ind_A, &h_val_A);
    free_host_csr(&h_row_ptr_B, &h_col_ind_B, &h_val_B);
    break;
}

```

```

int *h_row_ptr_D = (int *)malloc((size_t)(n + 1) * sizeof(int));
if (h_row_ptr_D == NULL) {

```

```

    fprintf(stderr, "Host allocation failed for D row ptr.\n");
    free_host_csr(&h_row_ptr_A, &h_col_ind_A, &h_val_A);
    free_host_csr(&h_row_ptr_B, &h_col_ind_B, &h_val_B);
    break;
}
for (int i = 0; i < n + 1; ++i) {
    h_row_ptr_D[i] = 0;
}

float *d_val_A = NULL;
int *d_row_ptr_A = NULL;
int *d_col_ind_A = NULL;
float *d_val_B = NULL;
int *d_row_ptr_B = NULL;
int *d_col_ind_B = NULL;
int *d_row_ptr_D = NULL;
int *d_col_ind_D = NULL;
float *d_val_D = NULL;
int nnz_D = 0;

size_t alloc_nnz_A = (size_t)(nnz_A > 0 ? nnz_A : 1);
size_t alloc_nnz_B = (size_t)(nnz_B > 0 ? nnz_B : 1);
HIP_CHECK(hipMalloc((void **)&d_val_A, alloc_nnz_A * sizeof(float)));
HIP_CHECK(hipMalloc((void **)&d_row_ptr_A, (size_t)(n + 1) * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_col_ind_A, alloc_nnz_A * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_val_B, alloc_nnz_B * sizeof(float)));
HIP_CHECK(hipMalloc((void **)&d_row_ptr_B, (size_t)(n + 1) * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_col_ind_B, alloc_nnz_B * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_row_ptr_D, (size_t)(n + 1) * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_col_ind_D, sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_val_D, sizeof(float)));

if (nnz_A > 0) {
    HIP_CHECK(hipMemcpy(d_val_A, h_val_A, (size_t)nnz_A * sizeof(float),
        hipMemcpyHostToDevice));
    HIP_CHECK(hipMemcpy(d_col_ind_A, h_col_ind_A, (size_t)nnz_A * sizeof(int),
        hipMemcpyHostToDevice));
}
HIP_CHECK(hipMemcpy(d_row_ptr_A, h_row_ptr_A, (size_t)(n + 1) * sizeof(int),
    hipMemcpyHostToDevice));

```



```

if (nnz_B > 0) {
    HIP_CHECK(hipMemcpy(d_val_B, h_val_B, (size_t)nnz_B * sizeof(float),
                        hipMemcpyHostToDevice));
    HIP_CHECK(hipMemcpy(d_col_ind_B, h_col_ind_B, (size_t)nnz_B * sizeof(int),
                        hipMemcpyHostToDevice));
}
HIP_CHECK(hipMemcpy(d_row_ptr_B, h_row_ptr_B, (size_t)(n + 1) * sizeof(int),
                    hipMemcpyHostToDevice));

HIP_CHECK(hipMemcpy(d_row_ptr_D, h_row_ptr_D, (size_t)(n + 1) * sizeof(int),
                    hipMemcpyHostToDevice));

rocsparse_mat_descr descr_A;
rocsparse_mat_descr descr_B;
rocsparse_mat_descr descr_C;
rocsparse_mat_descr descr_D;
ROCSPARSE_CHECK(rocsparse_create_mat_descr(&descr_A));
ROCSPARSE_CHECK(rocsparse_create_mat_descr(&descr_B));
ROCSPARSE_CHECK(rocsparse_create_mat_descr(&descr_C));
ROCSPARSE_CHECK(rocsparse_create_mat_descr(&descr_D));

rocsparse_mat_info info_C;
ROCSPARSE_CHECK(rocsparse_create_mat_info(&info_C));

const float alpha = 1.0f;
const float beta = 0.0f;
size_t buffer_size = 0;

ROCSPARSE_CHECK(rocsparse_scsrgemm_buffer_size(
    sparse_handle, rocsparse_operation_none, rocsparse_operation_none, n, n,
    n, &alpha, descr_A, nnz_A, d_row_ptr_A, d_col_ind_A, descr_B, nnz_B,
    d_row_ptr_B, d_col_ind_B, &beta, descr_D, nnz_D, d_row_ptr_D,
    d_col_ind_D, info_C, &buffer_size));

void *d_buffer = NULL;
HIP_CHECK(hipMalloc((void **)&d_buffer, buffer_size));

int *d_row_ptr_C = NULL;
HIP_CHECK(hipMalloc((void **)&d_row_ptr_C, (size_t)(n + 1) * sizeof(int)));

```

```

int nnz_C = 0;
ROCSPARSE_CHECK(rocsparse_csrgermm_nnz(
    sparse_handle, rocsparse_operation_none, rocsparse_operation_none, n, n,
    n, descr_A, nnz_A, d_row_ptr_A, d_col_ind_A, descr_B, nnz_B,
    d_row_ptr_B, d_col_ind_B, descr_D, nnz_D, d_row_ptr_D, d_col_ind_D,
    descr_C, d_row_ptr_C, &nnz_C, info_C, d_buffer));

int *d_col_ind_C = NULL;
float *d_val_C = NULL;
size_t alloc_nnz_C = (size_t)(nnz_C > 0 ? nnz_C : 1);
HIP_CHECK(hipMalloc((void **)&d_col_ind_C, alloc_nnz_C * sizeof(int)));
HIP_CHECK(hipMalloc((void **)&d_val_C, alloc_nnz_C * sizeof(float)));

hipEvent_t sparse_start, sparse_stop;
HIP_CHECK(hipEventCreate(&sparse_start));
HIP_CHECK(hipEventCreate(&sparse_stop));
HIP_CHECK(hipEventRecord(sparse_start));
ROCSPARSE_CHECK(rocsparse_scsrgemv(
    sparse_handle, rocsparse_operation_none, rocsparse_operation_none, n, n,
    n, &alpha, descr_A, nnz_A, d_val_A, d_row_ptr_A, d_col_ind_A, descr_B,
    nnz_B, d_val_B, d_row_ptr_B, d_col_ind_B, &beta, descr_D, nnz_D, d_val_D,
    d_row_ptr_D, d_col_ind_D, descr_C, d_val_C, d_row_ptr_C, d_col_ind_C,
    info_C, d_buffer));
HIP_CHECK(hipEventRecord(sparse_stop));
HIP_CHECK(hipEventSynchronize(sparse_stop));
float rocsparse_ms = 0.0f;
HIP_CHECK(hipEventElapsedTime(&rocsparse_ms, sparse_start, sparse_stop));

int *h_row_ptr_C = (int *)malloc((size_t)(n + 1) * sizeof(int));
int *h_col_ind_C = NULL;
float *h_val_C = NULL;
if (nnz_C > 0) {
    h_col_ind_C = (int *)malloc((size_t)nnz_C * sizeof(int));
    h_val_C = (float *)malloc((size_t)nnz_C * sizeof(float));
}
if (h_row_ptr_C == NULL || (nnz_C > 0 && h_col_ind_C == NULL) ||
    (nnz_C > 0 && h_val_C == NULL)) {
    fprintf(stderr, "Host allocation failed for C CSR.\n");
    free(h_row_ptr_C);
    free(h_col_ind_C);
}

```

```

    free(h_val_C);
    HIP_CHECK(hipEventDestroy(sparse_start));
    HIP_CHECK(hipEventDestroy(sparse_stop));
    HIP_CHECK(hipFree(d_val_A));
    HIP_CHECK(hipFree(d_row_ptr_A));
    HIP_CHECK(hipFree(d_col_ind_A));
    HIP_CHECK(hipFree(d_val_B));
    HIP_CHECK(hipFree(d_row_ptr_B));
    HIP_CHECK(hipFree(d_col_ind_B));
    HIP_CHECK(hipFree(d_row_ptr_D));
    HIP_CHECK(hipFree(d_col_ind_D));
    HIP_CHECK(hipFree(d_val_D));
    HIP_CHECK(hipFree(d_row_ptr_C));
    HIP_CHECK(hipFree(d_col_ind_C));
    HIP_CHECK(hipFree(d_val_C));
    HIP_CHECK(hipFree(d_buffer));
    ROCSPARSE_CHECK(rocsparse_destroy_mat_info(info_C));
    ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_A));
    ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_B));
    ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_C));
    ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_D));
    free(h_row_ptr_D);
    free_host_csr(&h_row_ptr_A, &h_col_ind_A, &h_val_A);
    free_host_csr(&h_row_ptr_B, &h_col_ind_B, &h_val_B);
    break;
}

HIP_CHECK(hipMemcpy(h_row_ptr_C, d_row_ptr_C, (size_t)(n + 1) * sizeof(int),
                    hipMemcpyDeviceToHost));
if (nnz_C > 0) {
    HIP_CHECK(hipMemcpy(h_col_ind_C, d_col_ind_C, (size_t)nnz_C * sizeof(int),
                        hipMemcpyDeviceToHost));
    HIP_CHECK(hipMemcpy(h_val_C, d_val_C, (size_t)nnz_C * sizeof(float),
                        hipMemcpyDeviceToHost));
}

csr_to_dense(n, h_row_ptr_C, h_col_ind_C, h_val_C, hC_sparse_dense);
float sparse_err = max_abs_diff(hC_sparse_dense, hC_ref, n * n);
int sparse_ok = (sparse_err <= tol) ? 1 : 0;

```

```

float *dA_dense = NULL;
float *dB_dense = NULL;
float *dC_dense = NULL;
HIP_CHECK(hipMalloc((void **)&dA_dense, (size_t)n * (size_t)n * sizeof(float)));
HIP_CHECK(hipMalloc((void **)&dB_dense, (size_t)n * (size_t)n * sizeof(float)));
HIP_CHECK(hipMalloc((void **)&dC_dense, (size_t)n * (size_t)n * sizeof(float)));
HIP_CHECK(hipMemcpy(dA_dense, hA, (size_t)n * (size_t)n * sizeof(float),
                    hipMemcpyHostToDevice));
HIP_CHECK(hipMemcpy(dB_dense, hB, (size_t)n * (size_t)n * sizeof(float),
                    hipMemcpyHostToDevice));

const float one = 1.0f;
const float zero = 0.0f;
hipEvent_t dense_start, dense_stop;
HIP_CHECK(hipEventCreate(&dense_start));
HIP_CHECK(hipEventCreate(&dense_stop));

HIP_CHECK(hipEventRecord(dense_start));
/* row-major GEMM via column-major routine:  $C^T = B^T * A^T$  */
ROCBLAS_CHECK(rocblas_sgemm(blas_handle, rocblas_operation_none,
                            rocblas_operation_none, n, n, n, &one, dB_dense,
                            n, dA_dense, n, &zero, dC_dense, n));
HIP_CHECK(hipEventRecord(dense_stop));
HIP_CHECK(hipEventSynchronize(dense_stop));
float rocblas_ms = 0.0f;
HIP_CHECK(hipEventElapsedTime(&rocblas_ms, dense_start, dense_stop));

HIP_CHECK(hipMemcpy(hC_dense, dC_dense, (size_t)n * (size_t)n * sizeof(float),
                    hipMemcpyDeviceToHost));
float dense_err = max_abs_diff(hC_dense, hC_ref, n * n);
int dense_ok = (dense_err <= tol) ? 1 : 0;

printf("%.8e,%.8e,%d,%d,%.6f,%.6f,%.6e,%.6e,%d,%d\n", density_percent,
        density_fraction, nnz_A, nnz_B, rocblas_ms, rocblas_ms, sparse_err,
        dense_err, sparse_ok, dense_ok);

HIP_CHECK(hipEventDestroy(sparse_start));
HIP_CHECK(hipEventDestroy(sparse_stop));
HIP_CHECK(hipEventDestroy(dense_start));
HIP_CHECK(hipEventDestroy(dense_stop));

```

```
HIP_CHECK(hipFree(d_val_A));
HIP_CHECK(hipFree(d_row_ptr_A));
HIP_CHECK(hipFree(d_col_ind_A));
HIP_CHECK(hipFree(d_val_B));
HIP_CHECK(hipFree(d_row_ptr_B));
HIP_CHECK(hipFree(d_col_ind_B));
HIP_CHECK(hipFree(d_row_ptr_D));
HIP_CHECK(hipFree(d_col_ind_D));
HIP_CHECK(hipFree(d_val_D));
HIP_CHECK(hipFree(d_row_ptr_C));
HIP_CHECK(hipFree(d_col_ind_C));
HIP_CHECK(hipFree(d_val_C));
HIP_CHECK(hipFree(d_buffer));
HIP_CHECK(hipFree(dA_dense));
HIP_CHECK(hipFree(dB_dense));
HIP_CHECK(hipFree(dC_dense));
```

```
ROCSPARSE_CHECK(rocsparse_destroy_mat_info(info_C));
ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_A));
ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_B));
ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_C));
ROCSPARSE_CHECK(rocsparse_destroy_mat_descr(descr_D));
```

```
free(h_row_ptr_C);
free(h_col_ind_C);
free(h_val_C);
free(h_row_ptr_D);
free_host_csr(&h_row_ptr_A, &h_col_ind_A, &h_val_A);
free_host_csr(&h_row_ptr_B, &h_col_ind_B, &h_val_B);
}
```

```
free(hA);
free(hB);
free(hC_ref);
free(hC_dense);
free(hC_sparse_dense);
```

```
ROCBLAS_CHECK(rocblas_destroy_handle(blas_handle));
ROCSPARSE_CHECK(rocsparse_destroy_handle(sparse_handle));
```

```

    HIP_CHECK(hipDeviceReset());

    return 0;
}

```

// Makefile

```

ROCM_PATH ?= /opt/rocm-6.3.3
CXX ?= g++
CXXFLAGS := -O2 -std=c++11 -D__HIP_PLATFORM_AMD__
INCLUDES := -I$(ROCM_PATH)/include
LIBPATH := -L$(ROCM_PATH)/lib
LIBS := -lamdhip64 -lrocblas -lrocspare
RPATH := -Wl,-rpath,$(ROCM_PATH)/lib

TARGET := P09
SRC := P09.cc
OUTPUT := p09_results.txt

.PHONY: all clean run

all: $(TARGET)

$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(SRC) $(INCLUDES) $(LIBPATH) $(LIBS) $(RPATH) -o
    $(TARGET)

run: $(TARGET)
    ./$(TARGET) > $(OUTPUT)

clean:
    rm -f $(TARGET) $(OUTPUT)

```

3) MATLAB Plot Script

```

data = readmatrix("p09_results.txt");

density_percent = data(:,1);
density_fraction = data(:,2);

nnz_a = data(:,3);
nnz_b = data(:,4);

```

```

rocsp_time = data(:,5);
rocblas_time = data(:,6);

x = density_fraction;

figure;

loglog(x, rocsp_time, '-o');
hold on;
loglog(x, rocblas_time, '-o');

xlabel("Density (fraction)");
ylabel("Time (ms)");
legend("rocSPARSE", "rocBLAS");
grid on;
title("SpGEMM vs Dense GEMM Performance");

```

4) Discussion

In P09, we are tasked with computing the multiplication of two matrices using varying densities by implementing both the rocSPARSE API and rocBLAS API functions, and then comparing their performances. Additionally, we are comparing both methods' performances by varying the densities from 10^{-7} to 10^0 in powers of 10 and plotting the elapsed time of each GEMM operation. After logging onto the Mantis server and specifically going into the nodeE shell, we develop a HIP program that conducts matrix multiplication using both of the methods. In order to run the program, I used a reliable method such as a Makefile to compile and run the code, producing a .txt with recorded runtime data to be graphed using a Matlab script. After producing the plot via a Matlab script, we can see that rocSPARSE is much faster at very low density and rocBLAS performs better at higher density. Additionally, we can see that there is a crossover between the two when density is 10^{-6} to align with the conceptual understanding in the previous sentence. Once again, the general consensus from the Matlab plot is that rocSPARSE is supposed to perform better at lower density while rocBLAS is supposed to perform better at higher densities. This is because rocSPARSE is able to skip zero operations making it faster at lower densities, and rocBLAS is better at higher densities because of optimized dense kernels. With that being said, the ninth programming assignment has introduced us to HIP programming and how to conduct matrix multiplication there.